SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

# StripeSense

*Detailed Project Report*

*Submitted by*

**Submitted by**
**Yashvardhan Samadhiya – 16010124058**
**Amey Salunkhe – 16010124057**
**Shrived Deshmukh – 16010124071**

B.Tech Computer Engineering
Third Year / Semester: V

Faculty: Shruti Javakar
Academic Year: 2026–2027

Mini Project: FSD-MERN Stack

GitHub Repository: N.A. (publication pending)

# Table of contents

1. Introduction
2. Requirements and Technology Stack
3. System Design
4. Application Design and Implementation
5. Testing and Results
6. GitHub and Deployment
7. Challenges and Individual Contributions
8. Conclusion and Future Scope
References

# List of figures

Figure 1: System architecture
Figure 2: Application workflow
Figure 3: Database design/schema
Figure 4: Landing page
Figure 5: Sign in screen
Figure 6: GitHub repository (N.A.)

# List of tables

Table 1: Functional Requirements
Table 2: Non-Functional Requirements
Table 3: MERN Technology Stack
Table 4: Modules and Workflow
Table 5: Database Design
Table 6: API Design
Table 7: Database and CRUD Implementation
Table 8: Test Cases
Table 9: Deployment Details
Table 10: Challenges and Solutions
Table 11: Individual Contributions

# 1. Introduction

## 1.1 Background

Urine reagent strips contain pads that change colour. Organising strip photographs with patient details and reference comparisons makes records easier to retrieve and review.

StripeSense is a MERN laboratory project that combines a React interface, Express APIs and MongoDB storage. It compares photographed pad colours with a supplied reference profile without a trained machine-learning model. Results are unvalidated colour estimates; users must always consult a doctor.

## 1.2 Problem Statement

Doctors and individuals need to keep strip photographs and results linked to the correct patient. Separate images and notes make retrieval difficult. StripeSense provides private patient records and saved reports in one web application.

## 1.3 Proposed Solution

Users sign in, add a patient, upload a ten-pad strip photograph and reference profile, select the pad row and submit it. Axios sends the request to Express. The backend compares colours and saves the image and result in MongoDB for later viewing and printing.

## 1.4 Objectives and Scope

- Provide registration, sign in, persistent sessions and sign out.
- Create patient records and associate each test with its patient and account owner.
- Process a ten-pad image using a supplied colour-reference profile.
- Save and retrieve reports, source images and analysis metadata in MongoDB.
- Demonstrate API integration, input validation, account isolation and automated testing.

Included: authentication, patient creation, colour matching, history, reports, deletion and dark mode. Hardware, trained ML, clinical validation, record editing and public deployment are outside the current scope.

# 2. REQUIREMENTS AND TECHNOLOGY STACK

## 2.1 Users and Functional Requirements

| ID | User/Function | Description |
|---|---|---|
| FR1 | Account access | Register, sign in/out and persist a private session. |
| FR2 | Patient directory | Add named patients with optional auto-generated IDs. |
| FR3 | Photo analysis | Upload a ten-pad photo/profile, crop and compare colours. |
| FR4 | History and reports | Retrieve account-owned tests, source images and printable reports. |
| FR5 | Privacy and deletion | Block other-account access and delete an owned test. |

Doctor and individual accounts have private workspaces. Account type changes wording; it does not grant access to another account's records.

## 2.2 Non-Functional Requirements

| Category | Requirement |
|---|---|
| Usability | Clear forms, desktop navigation, readable errors and persistent light/dark mode. |
| Performance | Bounded uploads/pixels and at most 500 rows per list request. No latency claim. |
| Reliability | Save complete results and reject invalid images/profiles before insertion. |
| Security | Password hashing, expiring sessions and owner-filtered queries. |
| Maintainability | Separate npm workspaces, reusable components and consistent Prettier formatting. |

## 2.3 MERN Technology Stack

| Layer | Technology | Purpose |
|---|---|---|
| Frontend | React.js, Axios, Vite | UI, routing and API requests |
| Backend | Node.js + Express.js, Sharp | APIs and colour matching |
| Database | MongoDB + native driver | Storage and indexed queries |
| API/Testing | Node.js test runner | Unit and integration tests |
| Version Control | N.A. | GitHub publication pending |
| Deployment | N.A. (local only) | No public hosting |

MongoDB uses the official Node.js driver; Mongoose is not used.

# 3. SYSTEM DESIGN

## 3.1 System Architecture

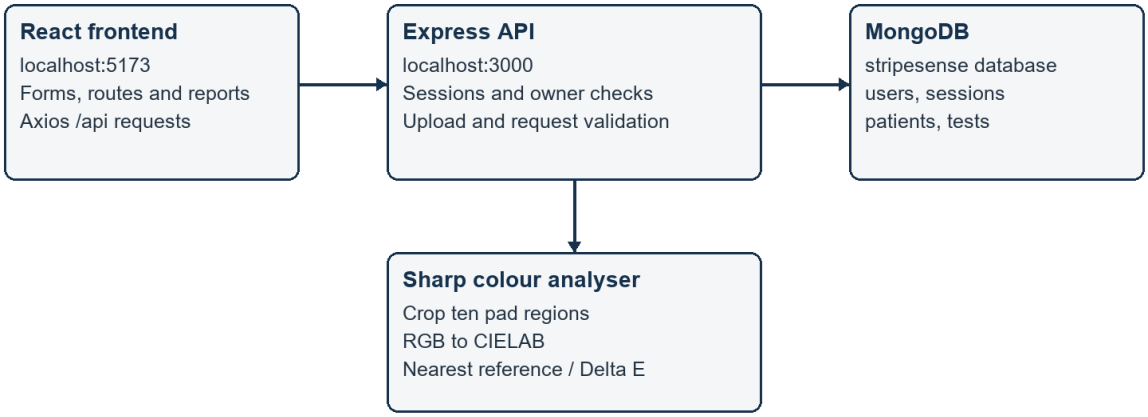



Figure 1: System architecture

## 3.2 Modules and Workflow

| Module | Main Functions |
|---|---|
| Authentication | Register/login/logout, current user, session cookies and expiry. |
| Patient records | Create/list owned patients and auto-generate IDs. |
| Colour analysis | Image/profile validation, crop sampling and nearest-reference matching. |
| Reporting and history | Save, filter, view, print and delete owned tests. |

Sign in → select patient → upload image/profile → select crop → analyse → save → review report.

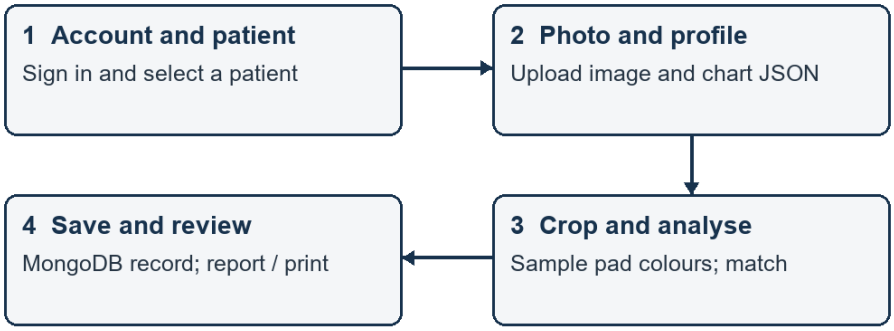



Figure 2: Application workflow

## 3.3 Database Design

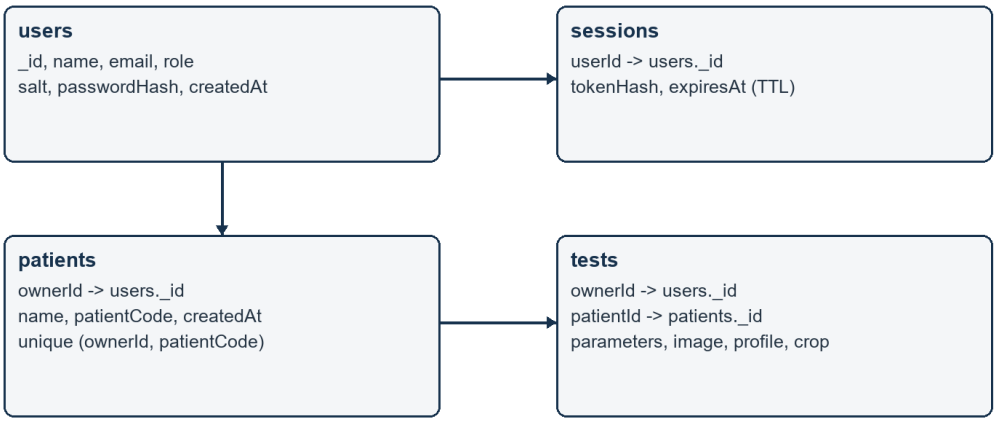



*Figure 3: Database design/schema*

| Collection | Important Fields | Purpose |
|---|---|---|
| users | _id, name, email, role, salt, passwordHash, createdAt | Account identity and password verification. |
| sessions | tokenHash, userId, expiresAt | Expiring account sessions. |
| patients | _id, ownerId, name, patientCode, createdAt | Private patient directory. |
| tests | ownerId, patientId, createdAt, parameters, image, crop, referenceProfile | Saved report, image and reproducibility metadata. |

Ownership links each patient and test to a user. Unique email and patient-code indexes prevent duplicates. Sessions have expiry indexes; reports store image and reference snapshots.

## 3.4 API Design

| Method | Endpoint | Purpose |
|---|---|---|
| POST | /api/auth/register | Create account and session. |
| POST | /api/auth/login | Authenticate and issue session cookie. |
| GET | /api/auth/me | Retrieve current account. |
| POST | /api/auth/logout | Revoke current session. |
| GET | /api/health | Check database availability. |
| GET | /api/reference-example | Retrieve sample profile; sign in required. |
| GET, POST | /api/patients | List or create owned patients. |
| GET | /api/tests?patientId=... | List/filter owned tests. |
| POST | /api/tests | Upload, analyse and save a test. |
| GET | /api/tests/:id | Retrieve an owned report. |
| GET | /api/tests/:id/image | Serve the saved PNG. |
| DELETE | /api/tests/:id | Delete owned test and embedded image. |

All patient and test routes require a session and enforce account ownership. Patient/test PUT or PATCH routes are not implemented.

# 4. APPLICATION DESIGN AND IMPLEMENTATION

## 4.1 User Interface and Navigation

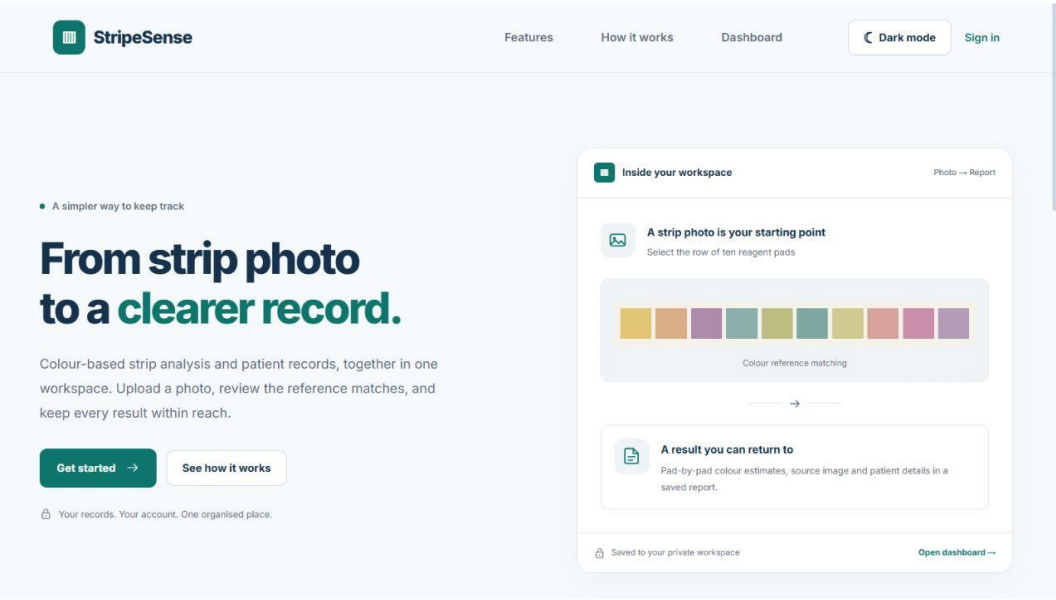



Figure 4: Landing page

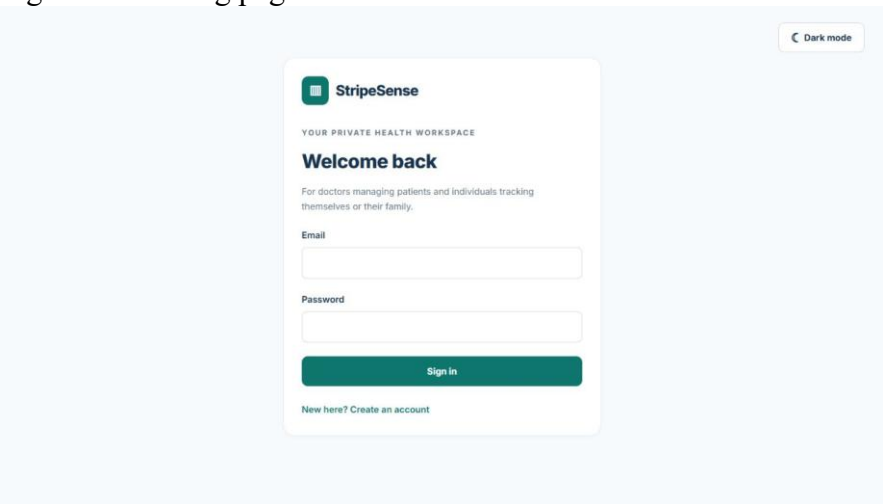



Figure 5: Sign in screen

## 4.2 Frontend Implementation

React components manage forms and loading states using hooks. React Router separates public and protected pages. AuthProvider loads the session; RequireAuth redirects signed-out users. Axios sends JSON and multipart requests through /api. ThemeProvider remembers light/dark mode.

## 4.3 Backend and API Implementation

Express validates requests and handles uploads through Multer. Sharp decodes images, samples ten crop regions and compares CIELAB colours using Delta E. Salted scrypt hashes protect passwords. HttpOnly session cookies and ownerId checks restrict access to private records.

## 4.4 Database and CRUD Implementation

| CRUD | Frontend Action | Backend/Database Action |
|---|---|---|
| Create | Add patient; submit analysis | POST endpoints; insertOne into patients/tests. |
| Read | Directory, dashboard, history, report | GET endpoints; find/findOne with ownerId. |
| Update | Not implemented for patient/test records | No PUT/PATCH route; planned enhancement. |
| Delete | Confirm deletion on saved report | DELETE owned test; deleteOne removes embedded image too. |

The native MongoDB driver stores users, sessions, patients and tests. insertOne creates records; find/findOne retrieves owned data; deleteOne removes an owned test. Images are embedded in test documents. Patient/test updates are not implemented.

## 4.5 Validation and Error Handling

Missing patient names and invalid crops/profiles are rejected. Duplicate patient codes return 409. Unsupported uploads produce validation errors. Signed-out API requests return 401; other-account reports return 404. The interface displays connection failures and flags near-tied colour matches for review.

# 5. TESTING AND RESULTS

## 5.1 Testing Approach

Node.js unit tests cover colour matching and interpretation. Integration tests exercise a real Express server and a temporary MongoDB database. On 30 October 2026, all eight tests passed without skips; npm run build also passed. Browser checks covered navigation, protected routes and themes.

## 5.2 Test Cases

| ID | Test Scenario | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| TC01 | Known pad colours | Correct match | All ten match | PASS |
| TC02 | Invalid crop/image/profile | Reject invalid input | Rejected | PASS |
| TC03 | Lab conversion | Expected lightness | Assertions pass | PASS |
| TC04 | Auth and storage | Persist / isolate | Checks pass | PASS |
| TC05 | Presentation labels | Map display text only | Stored labels preserved | PASS |
| TC06 | Estimate flags | Correct priority | Checks pass | PASS |
| TC07 | Unknown labels | No classification | Unknown returned | PASS |
| TC08 | Near-tied colours | Review only | Review group used | PASS |

Recorded automated checks use generated images and temporary data; they do not validate diagnostic accuracy.

## 5.3 API and Database Verification

The integration test verifies patient creation, upload, history, saved reports, image retrieval, deletion and account isolation. Database assertions confirm password hashing. MongoDB Compass connects to mongodb://127.0.0.1:27017; the local application database is stripesense. Postman was not used.

## 5.4 Final Results

The account, patient and upload-to-report workflows are implemented and tested. Reports display colour categories and Delta E, with a doctor-consultation disclaimer. Public deployment, record editing and clinical validation remain pending.

# 6. GITHUB AND DEPLOYMENT

## 6.1 GitHub Repository

**N.A. — GitHub repository publication is pending.**

*Figure 6: GitHub repository (N.A.)*

## 6.2 Deployment Details

| Item | Details |
|---|---|
| GitHub repository | N.A. - publication pending. |
| Frontend URL | http://localhost:5173 (local only) |
| Backend URL | http://localhost:3000 (local only) |
| Database | Local MongoDB; database stripesense |
| Deployment platform | N.A. - not publicly deployed. |

The application runs locally using npm run dev. No public deployment is available.

# 7. CHALLENGES AND INDIVIDUAL CONTRIBUTIONS

## 7.1 Challenges and Solutions

| Challenge | Solution | Outcome |
|---|---|---|
| Pad sampling initially read the wrong region | Materialise each Sharp crop before computing image statistics. | Known-colour tests pass for all ten pads. |
| Disconnected frontend/API processes | Use coordinated npm workspace scripts and a Vite /api proxy. | Both services start from npm run dev. |
| Private records needed account isolation | Session authentication plus ownerId in all protected queries. | Other-account report/image/delete requests are blocked. |

## 7.2 Individual Contributions

| Student | Frontend | Backend/API | Database | Testing/Deployment |
|---|---|---|---|---|
| Yashvardhan Samadhiya 16010124058 | — | Backend; supporting Python script | MongoDB integration | Not separately specified |
| Amey Salunkhe 16010124057 | Frontend | Supporting work | — | Not separately specified |
| Shrived Deshmukh 16010124071 | Frontend | Supporting work | — | Not separately specified |

Amey and Shrived developed the frontend with supporting backend work. Yashvardhan handled the backend, MongoDB integration and supporting Python scripting. Runtime image analysis uses Node.js and Sharp.

# 8. CONCLUSION AND FUTURE SCOPE

## 8.1 Conclusion

StripeSense integrates React, Axios, Express and MongoDB to manage private patient records and photograph-based colour comparisons. Users can save, retrieve, print and delete reports. Automated tests verify the implemented workflow and account isolation.

The project meets its MERN integration objective. Colour estimates are not validated medical measurements and cannot determine whether a result is clinically critical. Users must always consult a doctor.

## 8.2 Future Scope

Add patient/test editing, password reset, email verification, pagination and secure public deployment. Future analysis work could support manufacturer-specific reference charts and varied strip layouts, followed by independent validation before medical use.

# References

[1] React, “React documentation,” https://react.dev/.

[2] Express, “Express documentation,” https://expressjs.com/.

[3] MongoDB, “Node.js driver documentation,” https://www.mongodb.com/docs/drivers/node/current/.

[4] Axios, “Axios documentation,” https://axios-http.com/docs/intro.

[5] Sharp, “Sharp documentation,” https://sharp.pixelplumbing.com/.

[6] Node.js, “Node.js documentation,” https://nodejs.org/api/.